

03. Backend - Học Và Nhớ Library 74 Từ Gốc

Tài liệu này xem bạn là người mới học lập trình web ở mức cơ bản. Mục tiêu không phải học thuộc code, mà là hiểu backend đang giải quyết vấn đề gì, vì sao chọn cách đó, và khi hội đồng hỏi thì bạn giải thích được bằng logic.

Lộ trình học toàn project:

1. FE - Frontend: người dùng nhìn thấy gì, bấm gì, dữ liệu đi đâu.
2. DB - Database: dữ liệu được lưu như thế nào.
3. BE - Backend: nghiệp vụ, bảo mật, API.
4. AI - Recommendation, semantic search, xử lý tri thức.
5. Deploy/DevOps - đưa hệ thống chạy thật trên server.

Trong file này ta học phần BE.

1. Backend Là Gì?

1.1. Vấn Đề Thực Tế

Frontend chỉ là giao diện chạy trên trình duyệt. Nếu chỉ có frontend thì người dùng có thể nhìn thấy nút, form, bảng dữ liệu, nhưng hệ thống chưa thật sự biết:

- Ai đang đăng nhập.
- Người này có quyền mượn sách hay không.
- Quyền sách còn bản sao khả dụng hay không.
- Khi trả trễ thì phạt bao nhiêu.
- Email xác thực gửi như thế nào.
- Dữ liệu phải lưu vào bảng nào.
- Khi thủ thư xử lý một yêu cầu hỗ trợ thì ai được thông báo.

Backend là phần xử lý những câu hỏi đó.

Nói đơn giản:

```
Người dùng -> Frontend -> gọi API -> Backend -> Database / Kafka / AI /  
Email -> Backend trả kết quả -> Frontend hiển thị
```

1.2. Định Nghĩa

Backend là phần chạy trên server, chịu trách nhiệm:

- Nhận request từ frontend.
- Kiểm tra đăng nhập và phân quyền.
- Xử lý nghiệp vụ.
- Đọc/ghi database.

- Gọi hệ thống ngoài như AI, email, thanh toán.
- Trả response cho frontend.

Trong project Library74, backend nằm ở:

```
LMS_BE/
```

Nếu frontend là quầy giao tiếp với người dùng, thì backend là bộ phận xử lý hồ sơ phía sau.

2. Công Nghệ Backend Của Project

Project dùng các công nghệ chính:

Công nghệ	Vai trò
Java 21	Ngôn ngữ lập trình backend
Spring Boot	Framework tạo web API nhanh và chuẩn
Maven multi-module	Quản lý build và chia project thành nhiều module
Spring MVC	Nhận request HTTP qua Controller
Spring Security	Đăng nhập, JWT, phân quyền
OAuth2	Đăng nhập Google
PostgreSQL	Database chính
pgvector	Hỗ trợ vector/AI search ở database
JPA/Hibernate	Ánh xạ object Java với bảng database
Flyway	Quản lý version schema database
Kafka	Xử lý event bất đồng bộ
JavaMail/Zoho SMTP	Gửi email
PayOS	Thanh toán phí phạt
OpenAPI/Swagger	Tài liệu và test API
JUnit/Mockito/Testcontainers	Kiểm thử backend

3. Backend Nhận Request Như Thế Nào?

3.1. HTTP Request Là Gì?

Khi frontend gọi API, trình duyệt gửi một HTTP request.

Ví dụ:

```
GET /api/v1/contact-messages/my
Authorization: Bearer <access_token>
```

Ý nghĩa:

- **GET**: muốn lấy dữ liệu.
- **/api/v1/contact-messages/my**: đường dẫn API.
- **Authorization**: token chứng minh người dùng đã đăng nhập.

Backend xử lý xong sẽ trả HTTP response.

Ví dụ:

```
{
  "code": 200,
  "message": "Success",
  "data": [...]
}
```

3.2. Các HTTP Method Hay Gặp

Method	Ý nghĩa	Ví dụ trong Library74
GET	Lấy dữ liệu	Lấy danh sách sách, lấy phiếu hỗ trợ
POST	Tạo mới hoặc thực hiện hành động	Đăng nhập, tạo yêu cầu mượn
PUT	Cập nhật toàn bộ	Cập nhật hồ sơ
PATCH	Cập nhật một phần	Đổi trạng thái phiếu hỗ trợ
DELETE	Xóa	Xóa wishlist item, hủy đặt trước

Hội đồng có thể hỏi: "Vì sao không dùng toàn POST?"

Trả lời logic:

Mỗi HTTP method thể hiện ý nghĩa khác nhau. GET dùng để đọc, POST dùng để tạo hoặc thực hiện hành động, PATCH dùng để cập nhật một phần. Việc dùng đúng method giúp API rõ nghĩa, dễ test và dễ bảo trì.

4. Spring Boot Là Gì?

4.1. Vấn Đề Thực Tế

Nếu tự viết server từ đầu, ta phải tự xử lý rất nhiều việc:

- Mở cổng 8080.
- Parse JSON request.
- Gọi đúng hàm theo URL.
- Kết nối database.
- Quản lý transaction.
- Bảo mật token.
- Trả lỗi chuẩn.

Spring Boot giúp làm sẵn phần khung đó.

4.2. Định Nghĩa

Spring Boot là framework Java giúp xây dựng ứng dụng backend nhanh hơn bằng cách cung cấp sẵn cấu hình và thư viện phổ biến.

Trong project, ứng dụng chính nằm ở module:

```
LMS_BE/library-bootstrap/
```

Khi chạy backend, Spring Boot:

1. Đọc cấu hình trong `application.yml`.
2. Tạo các object cần thiết như Controller, Service, Repository.
3. Kết nối database, Kafka, mail.
4. Mở server HTTP ở cổng `8080`.
5. Nhận request từ frontend.

4.3. Annotation Là Gì?

Trong Java Spring, annotation là ký hiệu bắt đầu bằng `@`, dùng để nói với Spring một class hoặc method có vai trò gì.

Ví dụ:

```
@RestController
@RequestMapping("/api/v1/contact-messages")
public class ContactMessageController {
}
```

Ý nghĩa:

- `@RestController`: class này là nơi nhận API request và trả JSON.
- `@RequestMapping`: tất cả API trong class này bắt đầu bằng `/api/v1/contact-messages`.

Không cần học thuộc tất cả annotation, chỉ cần hiểu chúng là cách "đánh dấu vai trò" cho Spring.

5. Cấu Trúc Backend Library74

Backend được tổ chức theo Maven multi-module:

```
LMS_BE/  
├── library-bootstrap/  
├── library-shared/  
├── library-auth-module/  
├── library-user-module/  
├── library-catalog-module/  
├── library-circulation-module/  
└── library-recommendation-module/
```

5.1. Vì Sao Phải Chia Module?

Nếu tất cả code nằm trong một thư mục lớn, sau này rất khó hiểu:

- Code đăng nhập lẫn với code mượn sách.
- Code catalog lẫn với code phí phạt.
- Một sửa nhỏ dễ ảnh hưởng nhiều phần.

Chia module giúp mỗi nhóm nghiệp vụ có ranh giới rõ hơn.

Module	Vai trò
<code>library-bootstrap</code>	App chính, cấu hình, Flyway migrations
<code>library-shared</code>	Code dùng chung: response, exception, email, audit
<code>library-auth-module</code>	Đăng nhập, JWT, OAuth2, refresh token
<code>library-user-module</code>	User, role, admin/librarian management, contact inbox
<code>library-catalog-module</code>	Sách, bản sao, tác giả, danh mục, tag
<code>library-circulation-module</code>	Mượn, trả, đặt trước, phí phạt, chính sách
<code>library-recommendation-module</code>	Rating, wishlist, lịch sử tìm kiếm, AI gateway

Trả lời hội đồng:

Em chia backend theo nhóm nghiệp vụ để code dễ quản lý. Ví dụ catalog chỉ xử lý sách và bản sao, circulation chỉ xử lý mượn trả. Cách này giảm phụ thuộc lộn xộn và giúp mở rộng dễ hơn.

6. Luồng Cơ Bản: Controller -> Service/UseCase -> Repository -> Database

6.1. Tư Duy 4 Lớp

Một request backend thường đi qua các lớp:

Controller -> UseCase/Service -> Repository -> Database

Ý nghĩa:

Lớp	Làm gì
Controller	Nhận request, validate input, gọi use case
UseCase/Service	Xử lý nghiệp vụ
Repository	Đọc/ghi database
Entity	Đại diện bảng trong database

6.2. Ví Dụ Dễ Hiểu

Người dùng bấm "Tạo phiếu hỗ trợ":

```
Frontend gửi POST /api/v1/contact-messages  
-> ContactMessageController nhận request  
-> Kiểm tra người dùng đã đăng nhập  
-> Tạo ticket code dạng REQ-...  
-> Lưu vào bảng contact_messages  
-> Gửi notification cho thủ thư/admin  
-> Trả dữ liệu phiếu vừa tạo cho frontend
```

Người dùng không cần biết database có bảng gì. Frontend cũng không tự ghi database. Backend là nơi giữ luật nghiệp vụ.

6.3. Vì Sao Không Cho Frontend Gọi Thẳng Database?

Vì rất nguy hiểm:

- Lộ thông tin kết nối database.
- Người dùng có thể tự sửa dữ liệu.
- Không kiểm soát được quyền.
- Không kiểm tra được nghiệp vụ.

Trả lời hội đồng:

Frontend không được truy cập database trực tiếp vì trình duyệt là môi trường không tin cậy. Backend đứng giữa để xác thực, phân quyền, validate dữ liệu và bảo vệ database.

7. Controller Là Gì?

7.1. Vấn Đề Thực Tế

Frontend cần một "địa chỉ" để gửi yêu cầu.

Ví dụ:

- `/api/v1/auth/login`
- `/api/v1/publications`
- `/api/v1/borrowing-transactions`
- `/api/v1/contact-messages`

Controller chính là nơi định nghĩa các địa chỉ API đó.

7.2. Controller Trong Project

Một số controller quan trọng:

```
library-auth-module/.../AuthController.java
library-catalog-module/.../PublicationController.java
library-catalog-module/.../ItemController.java
library-circulation-module/.../BorrowingTransactionController.java
library-circulation-module/.../ReservationController.java
library-circulation-module/.../FineController.java
library-user-module/.../UserController.java
library-user-module/.../AdminManagementController.java
library-user-module/.../ContactMessageController.java
library-recommendation-module/.../RecommendationController.java
library-recommendation-module/.../AiSearchController.java
```

7.3. Controller Nên Làm Gì Và Không Nên Làm Gì?

Controller nên:

- Nhận request.
- Kiểm tra input cơ bản bằng validation.
- Lấy thông tin user hiện tại nếu cần.
- Gọi use case/service.
- Trả response.

Controller không nên:

- Chứa quá nhiều logic nghiệp vụ phức tạp.
- Tự viết nhiều câu SQL nếu có thể tách ra service/repository.
- Tự quyết định tất cả trạng thái nghiệp vụ.

Trong project vẫn có một số controller dùng `JdbcTemplate` trực tiếp cho tính năng nhanh như contact inbox. Khi bảo vệ, bạn có thể nói thật:

Với những phần nghiệp vụ lớn như mượn trả, project tách use case rõ. Với một số tính năng quản trị/contact cần truy vấn linh hoạt, project dùng JdbcTemplate trực tiếp để kiểm soát SQL và tối ưu tốc độ triển khai.

8. DTO, Request, Response Là Gì?

8.1. Vấn Đề Thực Tế

Database có thể có rất nhiều field nhạy cảm:

- `hashed_password`
- `provider_id`
- `token`
- thông tin nội bộ

Không thể trả nguyên Entity database ra frontend.

8.2. Định Nghĩa

DTO là object dùng để truyền dữ liệu giữa frontend và backend.

Có 2 loại thường gặp:

- Request DTO: dữ liệu frontend gửi lên.
- Response DTO: dữ liệu backend trả về.

Ví dụ trong contact:

```
SubmitContactMessageRequest  
ContactMessageResponse  
ContactMessageCommentResponse
```

Ý nghĩa:

- Request chỉ nhận các field cần thiết như `subject`, `message`, `category`.
- Response chỉ trả những field frontend cần hiển thị.

Trả lời hội đồng:

DTO giúp tách dữ liệu API khỏi Entity database. Nhờ vậy backend kiểm soát được field nào được nhận, field nào được trả về, tránh lộ dữ liệu nhạy cảm và giúp API ổn định hơn khi database thay đổi.

9. Validation Là Gì?

9.1. Vấn Đề Thực Tế

Người dùng có thể gửi dữ liệu sai:

- Email sai định dạng.
- Subject quá dài.
- Rating ngoài khoảng 1 đến 5.
- Message rỗng.
- Status không thuộc enum cho phép.

Frontend có thể validate trước, nhưng backend vẫn phải validate lại vì request có thể bị gửi bằng Postman/curl.

9.2. Validation Trong Project

Ví dụ trong `ContactMessageController`:

```
@NotBlank
@Size(max = 200)
String subject
```

Ý nghĩa:

- `@NotBlank`: không được rỗng.
- `@Size(max = 200)`: tối đa 200 ký tự.

Ví dụ status:

```
@Pattern(regexp = "NEW|IN_PROGRESS|RESOLVED|CLOSED")
String status
```

Ý nghĩa: chỉ cho phép các trạng thái hợp lệ.

Trả lời hội đồng:

Backend validate để bảo vệ hệ thống khỏi dữ liệu sai hoặc request giả mạo. Frontend validate giúp trải nghiệm tốt hơn, còn backend validate là lớp bảo vệ bắt buộc.

10. Database Và Entity

10.1. Entity Là Gì?

Entity là class Java đại diện cho một bảng trong database.

Ví dụ:

```
UserEntity -> bảng users
PublicationEntity -> bảng publications
ItemEntity -> bảng items
BorrowingTransactionEntity -> bảng borrowing_transactions
FineEntity -> bảng fines
ReservationEntity -> bảng reservations
RatingEntity -> bảng ratings
```

Entity giúp Java làm việc với dữ liệu dưới dạng object thay vì chỉ là dòng/cột SQL.

10.2. Những Nhóm Bảng Chính

Nhóm	Bảng tiêu biểu	Ý nghĩa
User/Auth	<code>users</code> , <code>roles</code> , <code>user_roles</code> , <code>refresh_tokens</code>	Tài khoản và phân quyền
Catalog	<code>publications</code> , <code>items</code> , <code>authors</code> , <code>categories</code> , <code>tags</code>	Dữ liệu sách
Circulation	<code>borrowing_transactions</code> , <code>reservations</code> , <code>fines</code>	Mượn trả, đặt trước, phạt
Recommendation	<code>ratings</code> , <code>wishlists</code> , <code>search_history</code>	Cá nhân hóa và tương tác
Contact	<code>contact_messages</code> , <code>contact_message_comments</code> , <code>contact_internal_notes</code>	Hỗ trợ người dùng
Technical	<code>flyway_schema_history</code>	Theo dõi migration database

10.3. Publication Và Item Khác Nhau Thế Nào?

Đây là câu rất hay bị hỏi.

```
Publication = đầu sách / metadata sách
Item = bản sao cụ thể có thể mượn
```

Ví dụ thư viện có sách "Clean Code".

- **Publication**: Clean Code, tác giả, ISBN, mô tả, ảnh bìa.
- **Item**: bản sao barcode `BC001`, `BC002`, `BC003`.

Một publication có nhiều item.

Trả lời hội đồng:

Em tách publication và item vì một đầu sách có thể có nhiều bản sao. Người dùng tìm kiếm theo publication, nhưng khi mượn thì hệ thống phải chọn một item cụ thể để theo dõi trạng thái AVAILABLE, BORROWED, RESERVED hoặc LOST.

11. Repository Là Gì?

11.1. Vấn Đề Thực Tế

Service cần đọc/ghi database, nhưng nếu service tự viết hết SQL thì code dễ rối.

Repository là lớp chuyên làm việc với database.

11.2. Repository Trong Project

Ví dụ:

```
UserJpaRepository  
PublicationJpaRepository  
ItemJpaRepository  
BorrowingTransactionJpaRepository  
FineJpaRepository  
ReservationJpaRepository  
RatingJpaRepository
```

Spring Data JPA có thể tự tạo query từ tên hàm.

Ví dụ logic:

```
findByEmail(...)  
findById(...)  
existsByEmail(...)
```

Ngoài JPA, project còn dùng **JdbcTemplate** khi cần query linh hoạt hoặc tối ưu.

Trả lời hội đồng:

Repository tách phần truy cập dữ liệu khỏi nghiệp vụ. Nhờ vậy service tập trung vào luật nghiệp vụ, còn repository tập trung vào cách lấy và lưu dữ liệu.

12. UseCase/Service Là Gì?

12.1. Vấn Đề Thực Tế

Một hành động nghiệp vụ thường không chỉ là một câu SQL.

Ví dụ "mượn sách":

1. Kiểm tra user có được mượn không.
2. Kiểm tra còn item khả dụng không.
3. Kiểm tra chính sách số sách tối đa.
4. Tạo transaction.
5. Đổi trạng thái item.
6. Gửi thông báo/email.

Nếu viết toàn bộ trong Controller thì Controller rất dài và khó test.

12.2. UseCase Trong Project

Project dùng nhiều class có hậu tố **UseCase**, ví dụ:

```
LoginUseCase  
VerifyEmailUseCase  
CreatePublicationUseCase  
BorrowRequestUseCase  
ReturnBookUseCase  
CreateReservationUseCase  
PayFineUseCase  
GetRecommendationsUseCase  
AddToWishlistUseCase
```

Mỗi UseCase đại diện cho một hành động nghiệp vụ cụ thể.

Trả lời hội đồng:

UseCase giúp biểu diễn rõ từng hành động của hệ thống. Ví dụ `ReturnBookUseCase` chỉ tập trung vào nghiệp vụ trả sách. Cách này giúp code dễ đọc, dễ test và dễ sửa hơn so với để mọi logic trong Controller.

13. Transaction Là Gì?

13.1. Vấn Đề Thực Tế

Một nghiệp vụ có thể cần cập nhật nhiều bảng cùng lúc.

Ví dụ trả sách:

- Cập nhật transaction thành **RETURNED**.
- Cập nhật item thành **AVAILABLE**.
- Nếu trễ hạn thì tạo fine.
- Có thể gửi notification.

Nếu cập nhật transaction xong nhưng cập nhật item fail, dữ liệu sẽ sai.

13.2. Định Nghĩa

Transaction là cơ chế đảm bảo một nhóm thao tác database hoặc thành công hết, hoặc thất bại thì rollback hết.

Trong Spring, thường dùng:

```
@Transactional
```

Ý nghĩa:

Nếu một bước trong nghiệp vụ lỗi, database quay lại trạng thái trước đó.

Trả lời hội đồng:

Transaction giúp đảm bảo tính nhất quán dữ liệu. Với nghiệp vụ mượn trả, một hành động có thể ảnh hưởng nhiều bảng, nên cần transaction để tránh trạng thái nửa thành công nửa thất bại.

14. Flyway Là Gì?

14.1. Vấn Đề Thực Tế

Database thay đổi theo thời gian:

- Thêm bảng contact.
- Thêm cột `librarian_campus`.
- Thêm index tìm kiếm.
- Seed dữ liệu demo.

Nếu mỗi người tự sửa database bằng tay thì môi trường local, test, production sẽ lệch nhau.

14.2. Định Nghĩa

Flyway là công cụ quản lý version database bằng file migration SQL.

Trong project, migration nằm ở:

```
LMS_BE/library-bootstrap/src/main/resources/db/migration/
```

Ví dụ:

```
V1__create_schema.sql  
V20__schema_cleanup_and_ai_indexes.sql  
V35__contact_messages.sql  
V36__contact_ticket_threads.sql  
V39__contact_ticket_notifications.sql  
V42__demo_slide_account_roles.sql
```

Khi backend khởi động, Flyway kiểm tra bảng `flyway_schema_history` để biết migration nào đã chạy rồi.

14.3. Vì Sao Không Dùng Hibernate Tự Tạo Bảng?

Trong production, project dùng:

```
spring.jpa.hibernate.ddl-auto=validate
```

Ý nghĩa:

- Hibernate chỉ kiểm tra schema có khớp Entity không.
- Không tự ý sửa database production.

Trả lời hội đồng:

Project dùng Flyway để mọi thay đổi schema có lịch sử rõ ràng và chạy giống nhau ở các môi trường. Hibernate chỉ validate để tránh tự động sửa database production ngoài kiểm soát.

15. Authentication Và Authorization

15.1. Authentication Là Gì?

Authentication là xác thực danh tính: "Bạn là ai?"

Ví dụ:

- Đăng nhập bằng email/password.
- Đăng nhập bằng Google OAuth2.
- Verify email.
- Refresh access token.

Module liên quan:

```
library-auth-module/
```

15.2. Authorization Là Gì?

Authorization là phân quyền: "Bạn được làm gì?"

Ví dụ:

- Student được xem sách, mượn sách, xem phiếu hỗ trợ của mình.
- Librarian được xử lý mượn trả, quản lý bản sao, xử lý contact inbox.
- Admin được quản lý tài khoản, role, audit.

15.3. JWT Là Gì?

JWT là token backend cấp sau khi đăng nhập. Frontend lưu token và gửi kèm mỗi request.

Luồng:

```
User login -> Backend kiểm tra password -> Backend cấp access token -> Frontend gọi API kèm Bearer token -> Backend xác minh token
```

15.4. Vì Sao Cần Refresh Token?

Access token nên sống ngắn để giảm rủi ro nếu bị lộ. Refresh token dùng để lấy access token mới mà không bắt user đăng nhập lại liên tục.

Trả lời hội đồng:

```
Access token dùng cho request hằng ngày và nên hết hạn nhanh. Refresh token sống lâu hơn để cấp access token mới. Cách này cân bằng giữa bảo mật và trải nghiệm người dùng.
```

16. Role Và Security Annotation

Project có các vai trò chính:

```
ADMIN  
LIBRARIAN  
STUDENT
```

Một số API có annotation bảo vệ:

```
@RequiresAuthentication  
@RequiresRole(...)  
@RequiresAnyRole(...)
```

Ý nghĩa:

- `@RequiresAuthentication`: phải đăng nhập.
- `@RequiresRole`: phải có đúng role.
- `@RequiresAnyRole`: có một trong các role được phép.

Ví dụ contact:

- User chỉ xem phiếu hỗ trợ của mình.
- Librarian xử lý inbox.
- Admin có thể xem ở chế độ quản trị.

Trả lời hội đồng:

Backend không tin frontend tự ẩn nút là đủ. Dù frontend không hiển thị nút admin, người dùng vẫn có thể gọi API bằng công cụ ngoài. Vì vậy backend phải kiểm tra role ở server.

17. Luồng Đăng Ký, Verify Email, Reset Password

17.1. Đăng Ký

Luồng logic:

```
Frontend gửi thông tin đăng ký
-> Backend kiểm tra email đã tồn tại chưa
-> Hash password
-> Tạo user
-> Gán role STUDENT
-> Gửi email verify
```

17.2. Verify Email

Luồng logic:

```
User bấm link email
-> Backend kiểm tra token verify
-> Nếu hợp lệ thì cập nhật user is_verified = true
```

17.3. Reset Password

Luồng logic:

```
User nhập email quên mật khẩu
-> Backend tạo reset token
-> Gửi email reset
-> User bấm link và nhập password mới
-> Backend kiểm tra token
-> Hash password mới và lưu
```

17.4. Vì Sao Phải Hash Password?

Backend không lưu password gốc. Nó lưu password đã hash.

Nếu database bị lộ, hacker không thấy trực tiếp password thật.

Trả lời hội đồng:

```
Project hash password trước khi lưu để bảo vệ người dùng. Khi đăng nhập,
backend hash password nhập vào rồi so sánh với bản hash trong database, chứ
không lưu hoặc so sánh password gốc.
```

18. Catalog: Quản Lý Sách Và Bản Sao

18.1. Nghiệp Vụ Catalog Giải Quyết Gì?

Catalog là phần quản lý tri thức thư viện:

- Ấn phẩm.
- Tác giả.
- Nhà xuất bản.
- Danh mục.
- Tag.
- Bản sao sách.
- Ảnh bìa.
- File PDF.

Module:

```
library-catalog-module/
```

18.2. Luồng Tạo Sách

Logic:

```
Thủ thư nhập metadata sách
-> Backend validate
```

```
-> Lưu publication  
-> Liên kết author/category/tag  
-> Nếu có ảnh/PDF thì lưu URL  
-> Frontend hiển thị sách mới
```

18.3. Tìm Kiếm Sách

Tìm kiếm có thể dựa trên:

- Tiêu đề.
- Tác giả.
- Danh mục.
- Tag.
- Mô tả.
- AI summary.
- Unaccent để hỗ trợ tiếng Việt không dấu.

Trả lời hội đồng:

Catalog là nền tảng cho toàn hệ thống. Mượn trả, recommendation và search đều dựa trên dữ liệu sách chuẩn trong catalog.

19. Circulation: Mượn, Trả, Đặt Trước, Phí Phạt

19.1. Circulation Là Gì?

Circulation là nghiệp vụ vận hành thư viện:

- Sinh viên yêu cầu mượn.
- Thủ thư xác nhận lấy sách.
- Sinh viên trả sách.
- Hệ thống tính quá hạn.
- Sinh viên thanh toán phí phạt.
- Đặt trước sách khi chưa có bản sao khả dụng.

Module:

```
library-circulation-module/
```

19.2. Các Trạng Thái Cần Hiểu

Item có thể có trạng thái:

```
AVAILABLE
BORROWED
RESERVED
IN_MAINTENANCE
LOST
```

Borrowing transaction có thể có trạng thái như:

```
REQUESTED
APPROVED
BORROWED
RETURNED
CANCELLED
OVERDUE
```

Tên trạng thái cụ thể tùy enum trong code, nhưng ý tưởng là: backend luôn phải biết giao dịch đang ở bước nào.

19.3. Luồng Mượn Sách

Logic:

```
Student bấm mượn
-> Backend kiểm tra đăng nhập
-> Kiểm tra credit score / chính sách / số sách đang mượn
-> Tìm item còn AVAILABLE
-> Tạo borrowing transaction
-> Đổi trạng thái item
-> Gửi thông báo/email nếu cần
```

19.4. Luồng Trả Sách

Logic:

```
Thủ thư quét/tra giao dịch
-> Backend tìm transaction đang active
-> Cập nhật ngày trả
-> Nếu quá hạn hoặc mất/hỏng thì tạo fine
-> Đổi item về AVAILABLE hoặc trạng thái phù hợp
```

Trả lời hội đồng:

Mượn trả là nghiệp vụ cần tính nhất quán cao, vì nó ảnh hưởng cùng lúc đến transaction, item và fine. Backend xử lý tập trung để tránh việc một bản sao bị mượn trùng hoặc trạng thái sai.

20. Fine Và PayOS

20.1. Fine Là Gì?

Fine là phí phạt khi:

- Trả sách trễ.
- Làm mất sách.
- Làm hỏng sách.
- Vi phạm chính sách mượn trả.

Backend lưu fine để student xem và thanh toán.

20.2. PayOS Trong Project

PayOS dùng để tạo link/thông tin thanh toán.

Luồng logic:

```
Student chọn thanh toán fine
-> Backend tạo payment order với PayOS
-> Frontend chuyển người dùng tới thanh toán
-> PayOS callback/webhook về backend
-> Backend cập nhật trạng thái fine đã thanh toán
```

Trả lời hội đồng:

Backend không tự xử lý tiền trực tiếp. Backend tích hợp PayOS để tạo giao dịch và nhận kết quả thanh toán qua webhook, sau đó cập nhật trạng thái phí phạt trong hệ thống.

21. Contact Inbox: Phiếu Hỗ Trợ

21.1. Vấn Đề Thực Tế

Người dùng cần liên hệ thư viện khi:

- Lỗi hệ thống.
- Vấn đề tài khoản.
- Vấn đề mượn trả.

- Gợi ý sách.
- Yêu cầu hỗ trợ khác.

Nếu chỉ gửi form contact bình thường, người dùng không theo dõi được tiến độ. Vì vậy project có contact ticket/inbox.

21.2. Luồng Người Dùng Gửi Phiếu

```
User tạo phiếu hỗ trợ
-> Backend sinh mã REQ-...
-> Lưu contact_messages
-> Tạo notification cho librarian/admin
-> User theo dõi ở trang contact-tickets
```

21.3. Luồng Thủ Thư Xử Lý

```
Librarian mở Contact Inbox
-> Xem danh sách phiếu
-> Reply / ghi chú nội bộ / đổi trạng thái
-> Khi bấm Đã xử lý, backend gửi email hoàn tất hỗ trợ
```

Email hiện tại:

Loại mail	From
Verify, reset password, mượn trả, phạt, đặt trước	noreply@library74.uk
Thủ thư bấm Đã xử lý ở contact inbox	support@library74.uk

Trả lời hội đồng:

Contact inbox biến form liên hệ thành một quy trình hỗ trợ có trạng thái. Người dùng biết yêu cầu của mình đang mới, đang xử lý hay đã hoàn tất. Thủ thư có thể phản hồi, ghi chú nội bộ và hệ thống gửi email hoàn tất từ địa chỉ support chuyên biệt.

22. Email Service

22.1. Vì Sao Backend Gửi Email?

Email dùng cho các thao tác cần thông báo ngoài hệ thống:

- Xác thực tài khoản.
- Quên mật khẩu.
- Thông báo mượn/trả/đặt trước/phạt.

- Hoàn tất hỗ trợ.

22.2. Cấu Hình Mail

Backend dùng SMTP Zoho:

```
MAIL_HOST=smtp.zoho.com
MAIL_SENDER_USERNAME=admin@library74.uk
```

Trong code, `EmailServiceImpl` có hai kiểu gửi:

- Gửi bình thường bằng `noreply@library74.uk`.
- Gửi hỗ trợ bằng `support@library74.uk`.

22.3. Vì Sao Tách noreply Và support?

`noreply` dùng cho thư hệ thống tự động, người dùng không cần trả lời.

`support` dùng cho thư hỗ trợ, có tính chất dịch vụ khách hàng hơn.

Trả lời hội đồng:

Project tách địa chỉ gửi mail theo ngữ cảnh. Các mail hệ thống như verify hoặc mượn trả gửi từ noreply, còn mail báo hoàn tất hỗ trợ gửi từ support để người dùng nhận diện đây là luồng chăm sóc/hỗ trợ.

23. Kafka Là Gì?

23.1. Vấn Đề Thực Tế

Không phải việc gì cũng nên xử lý ngay trong request.

Ví dụ khi user đăng ký:

- Tạo user là việc chính.
- Gửi email verify là việc phụ.

Nếu gửi email chậm, request đăng ký cũng bị chậm.

Kafka giúp tách những việc phụ thành event.

23.2. Định Nghĩa

Kafka là hệ thống message/event queue.

Luồng đơn giản:

```
Backend nghiệp vụ tạo event -> Kafka topic -> Consumer nhận event -> xử lý email/notification
```

Project có các topic như:

```
library.email  
notification.send  
user.registered  
user.forgot-password
```

23.3. Lợi Ích

- Request chính nhanh hơn.
- Xử lý email/notification tách riêng.
- Nếu cần mở rộng, có thể thêm consumer.
- Giảm phụ thuộc trực tiếp giữa các phần.

Trả lời hội đồng:

Kafka giúp backend xử lý bất đồng bộ. Những việc không cần trả kết quả ngay như gửi email hoặc notification có thể đưa vào queue, giúp request chính nhẹ hơn và hệ thống dễ mở rộng.

24. Notification

Notification khác email ở chỗ:

- Email đi ra ngoài hộp thư.
- Notification hiển thị trong app.

Ví dụ:

- Có phiếu hỗ trợ mới.
- Yêu cầu mượn được duyệt.
- Sách đến hạn trả.
- Có review mới.

Backend lưu notification để frontend lấy và hiển thị.

Trả lời hội đồng:

Email phù hợp cho thông báo quan trọng ngoài hệ thống, còn notification phù hợp cho trải nghiệm trong app. Project dùng cả hai để người dùng không bỏ

lỗi thông tin.

25. AI Gateway Trong Backend

25.1. Backend Có Tự Làm AI Không?

Trong project này, backend Java không trực tiếp chạy model AI.

Backend gọi sang AI service:

```
Backend -> AI Gateway/FastAPI -> xử lý AI -> trả kết quả/callback
```

Module liên quan:

```
library-recommendation-module/
```

Class quan trọng:

```
AiGatewayService  
AiSearchController  
AiCallbackController  
RecommendationController  
SimilarPublicationsController
```

25.2. Backend Chịu Trách Nhiệm Gì Với AI?

Backend chịu trách nhiệm:

- Nhận request search/recommendation từ frontend.
- Kiểm tra user và quyền.
- Gọi AI service đúng endpoint.
- Lưu hoặc cập nhật kết quả AI vào database nếu cần.
- Nhận callback khi AI xử lý PDF/metadata xong.

Trả lời hội đồng:

```
Backend đóng vai trò điều phối giữa frontend, database và AI service. AI  
service chuyên xử lý model và vector, còn backend giữ nghiệp vụ, bảo mật và  
dữ liệu chính.
```

26. Exception Và Response Chuẩn

26.1. Vấn Đề Thực Tế

Nếu mỗi API trả lỗi một kiểu, frontend rất khó xử lý.

Ví dụ lỗi có thể là:

- Không đăng nhập.
- Không có quyền.
- Không tìm thấy sách.
- Dữ liệu không hợp lệ.
- Lỗi server.

26.2. Response Chuẩn

Project dùng response dạng chung:

```
{
  "code": 200,
  "message": "Success",
  "data": {}
}
```

Khi lỗi:

```
{
  "code": 1400,
  "message": "Authentication required"
}
```

Lớp xử lý lỗi chung thường nằm ở shared exception handler.

Trả lời hội đồng:

Response chuẩn giúp frontend xử lý thống nhất. Khi có lỗi, frontend chỉ cần đọc code/message thay vì đoán nhiều format khác nhau.

27. Configuration Và Environment Variables

27.1. Vì Sao Không Hardcode Secret?

Không nên viết thẳng secret vào code:

- Mật khẩu mail.
- OAuth client secret.
- PayOS key.

- AWS key.
- Database password.

Vì code có thể bị commit/push lên Git.

27.2. application.yml

File cấu hình chính:

```
LMS_BE/library-bootstrap/src/main/resources/application.yml
```

Ví dụ logic:

```
spring:
  mail:
    host: ${MAIL_HOST:smtp.zoho.com}
    username: ${MAIL_SENDER_USERNAME:}
    password: ${MAIL_SENDER_PASSWORD:}
```

Ý nghĩa:

- Nếu có biến môi trường **MAIL_HOST**, dùng biến đó.
- Nếu không có, dùng default sau dấu **:**.

27.3. .env.prod

Production dùng **.env.prod** để truyền secret cho Docker Compose.

Trả lời hội đồng:

Project tách cấu hình khỏi code. Code chỉ đọc biến môi trường, còn giá trị thật nằm trong file env trên server. Cách này an toàn hơn và giúp cùng một code chạy được ở local, test và production.

28. OpenAPI/Swagger

Swagger UI giúp xem và test API trực tiếp trên trình duyệt.

Local thường có:

```
http://localhost:8080/swagger-ui/index.html
```

Lợi ích:

- Biết backend có endpoint nào.
- Xem request/response schema.
- Test nhanh API không cần frontend.
- Hỗ trợ báo cáo đồ án.

Trả lời hội đồng:

Swagger giúp document API tự động từ backend, làm cầu nối giữa frontend và backend. Frontend có thể dựa vào đó để biết phải gọi endpoint nào và truyền dữ liệu gì.

29. Testing Backend

29.1. Vì Sao Cần Test?

Backend chứa nghiệp vụ quan trọng. Nếu sai:

- User có thể mượn quá số lượng.
- Sách có thể bị mượn trùng.
- Fine có thể tính sai.
- API có thể lộ dữ liệu.

29.2. Các Công Cụ Test

Công cụ	Vai trò
JUnit 5	Viết test Java
Mockito	Mock dependency
Spring Boot Test	Test với Spring context
Testcontainers	Chạy database thật trong Docker cho test

29.3. Lệnh Test

Chạy toàn bộ test:

```
cd LMS_BE
mvn test
```

Compile nhanh bỏ qua test:

```
mvn -q -DskipTests compile
```

Trả lời hội đồng:

Backend cần test vì nó giữ nghiệp vụ lỗi. Test giúp phát hiện lỗi sớm, nhất là các luồng nhiều bước như mượn trả, đặt trước, thanh toán và phân quyền.

30. Deploy Backend

30.1. Backend Chạy Ở Production Như Thế Nào?

Production dùng Docker Compose.

Backend chạy trong container:

```
lms_backend
```

Các service liên quan:

```
lms_postgres  
lms_kafka  
lms_rabbitmq  
lms_ai_api  
lms_ai_worker  
lms_caddy  
lms_frontend
```

30.2. Caddy Là Gì?

Caddy là reverse proxy nhận request từ internet:

```
https://library74.uk/api/... -> Caddy -> backend:8080  
https://library74.uk/... -> Caddy -> frontend
```

30.3. Healthcheck

Backend có healthcheck:

```
docker exec lms_backend wget -qO- http://localhost:8080/actuator/health
```

Kỳ vọng:

```
{"status": "UP"}
```

Trả lời hội đồng:

Production dùng Docker Compose để chạy các service độc lập nhưng cùng network. Caddy đứng ngoài nhận HTTPS, sau đó proxy API về backend và giao diện về frontend.

31. Một Request Thực Tế Đi Qua Hệ Thống Như Thế Nào?

Ví dụ: thủ thư bấm **Đã xử lý** trong Contact Inbox.

1. Frontend gửi PATCH /api/v1/contact-messages/{id}
2. Backend kiểm tra token và role LIBRARIAN/ADMIN
3. Backend validate status là RESOLVED
4. Backend cập nhật contact_messages
5. Backend lưu reply/internal note nếu có
6. Backend gửi notification trong app
7. Backend gửi email hoàn tất hỗ trợ từ support@library74.uk
8. Backend trả response
9. Frontend cập nhật UI sang trạng thái Đã xử lý

Nếu hội đồng hỏi "frontend hay backend quyết định phiếu đã xử lý?", trả lời:

Frontend chỉ gửi yêu cầu và hiển thị kết quả. Backend mới là nơi quyết định hợp lệ hay không, vì backend kiểm tra quyền, validate trạng thái và cập nhật database.

32. Những Câu Hội Đồng Hay Hỏi

Câu 1: Vì sao cần backend khi đã có frontend?

Trả lời:

Frontend chỉ chạy trên trình duyệt và không đáng tin cậy để giữ nghiệp vụ hoặc secret. Backend xử lý đăng nhập, phân quyền, nghiệp vụ, database và tích hợp hệ thống ngoài. Nếu không có backend, dữ liệu và bảo mật sẽ không được kiểm soát.

Câu 2: Vì sao dùng Spring Boot?

Trả lời:

Spring Boot cung cấp sẵn nền tảng cho REST API, dependency injection, security, database, transaction và cấu hình. Với hệ thống nhiều nghiệp vụ như thư viện, Spring Boot giúp backend có cấu trúc rõ và dễ mở rộng.

Câu 3: Vì sao chia multi-module?

Trả lời:

Project chia module theo nghiệp vụ: auth, user, catalog, circulation, recommendation. Cách này giúp mỗi phần có trách nhiệm rõ, giảm code lẫn lộn và dễ bảo trì hơn khi hệ thống lớn.

Câu 4: Controller khác Service khác Repository thế nào?

Trả lời:

Controller nhận request và trả response. Service hoặc UseCase xử lý nghiệp vụ. Repository làm việc với database. Tách như vậy giúp code rõ trách nhiệm và dễ test.

Câu 5: Vì sao dùng Flyway?

Trả lời:

Flyway quản lý schema database bằng migration có version. Nhờ vậy local, test và production chạy cùng một lịch sử thay đổi database, tránh sửa tay gây lệch môi trường.

Câu 6: JWT dùng để làm gì?

Trả lời:

JWT là token chứng minh người dùng đã đăng nhập. Frontend gửi token trong header, backend kiểm tra token để biết user là ai và có role gì trước khi cho gọi API.

Câu 7: Vì sao không chỉ ẩn nút trên frontend để bảo mật?

Trả lời:

Ấn nút chỉ cải thiện giao diện, không phải bảo mật. Người dùng vẫn có thể gọi API trực tiếp bằng công cụ ngoài. Vì vậy backend phải kiểm tra token và role cho từng API quan trọng.

Câu 8: Vì sao publication và item tách riêng?

Trả lời:

Publication là đầu sách, còn item là từng bản sao cụ thể. Một đầu sách có thể có nhiều bản sao, mỗi bản sao có barcode và trạng thái riêng. Tách như vậy giúp quản lý mượn trả chính xác.

Câu 9: Kafka dùng để làm gì?

Trả lời:

Kafka dùng để xử lý bất đồng bộ các event như gửi email và notification. Request chính không bị phụ thuộc trực tiếp vào thời gian gửi email, giúp hệ thống phản hồi nhanh và dễ mở rộng.

Câu 10: Backend liên kết AI như thế nào?

Trả lời:

Backend không trực tiếp chạy model AI. Backend nhận request từ frontend, kiểm tra quyền và gọi AI service qua AI Gateway. AI service xử lý semantic search/recommendation rồi trả kết quả hoặc callback về backend.

33. Cách Tự Học Code Backend Trong Project

Không nên mở tất cả file cùng lúc. Học theo luồng.

33.1. Luồng Đăng Nhập

Đọc theo thứ tự:

```
AuthController
LoginUseCase
AuthServiceImpl
UserDetailsServiceImpl
RefreshTokensServiceImpl
```

Câu hỏi cần trả lời:

- Request login gồm gì?
- Backend kiểm tra password ở đâu?
- Token được tạo ở đâu?
- Refresh token lưu ở đâu?

33.2. Luồng Tìm Kiếm Sách

Đọc theo thứ tự:

```
PublicationController  
SearchPublicationsUseCase  
PublicationJpaRepository / PublicationRepositoryImpl  
PublicationEntity
```

Câu hỏi cần trả lời:

- Frontend gửi keyword/filter gì?
- Backend query những bảng nào?
- Response trả về có đủ thông tin card sách không?

33.3. Luồng Mượn Trả

Đọc theo thứ tự:

```
BorrowingTransactionController  
BorrowRequestUseCase  
ConfirmPickupUseCase  
ReturnBookUseCase  
BorrowingTransactionEntity  
ItemEntity  
FineEntity
```

Câu hỏi cần trả lời:

- Khi nào item đổi trạng thái?
- Khi nào tạo fine?
- Ai có quyền xác nhận mượn/trả?

33.4. Luồng Contact Inbox

Đọc theo thứ tự:

```
ContactMessageController  
EmailService  
EmailServiceImpl
```

```
EmailTemplates
V35__contact_messages.sql
V36__contact_ticket_threads.sql
V38__contact_internal_notes.sql
V39__contact_ticket_notifications.sql
```

Câu hỏi cần trả lời:

- Phiếu hỗ trợ lưu ở bảng nào?
- Comment và internal note khác nhau thế nào?
- Khi nào gửi mail từ support?

34. Bản Tóm Tắt Một Phút

Nếu phải giải thích backend Library74 trong một phút:

Backend Library74 là hệ thống Spring Boot Java 21 chạy trên server, chịu trách nhiệm xử lý nghiệp vụ thư viện, bảo mật và dữ liệu. Frontend chỉ gửi request, còn backend kiểm tra token JWT, phân quyền theo role STUDENT/LIBRARIAN/ADMIN, validate input, xử lý nghiệp vụ và lưu PostgreSQL.

Project được chia multi-module theo nghiệp vụ: auth cho đăng nhập, user cho tài khoản và contact, catalog cho sách và bản sao, circulation cho mượn trả/phí phạt, recommendation cho rating/wishlist/AI gateway. Database được quản lý bằng Flyway migration để production và local nhất quán.

Backend còn tích hợp Kafka để xử lý email/notification bất đồng bộ, Zoho SMTP để gửi email, PayOS cho thanh toán phí phạt, và AI service để semantic search/recommendation. Production chạy bằng Docker Compose, Caddy reverse proxy HTTPS về frontend và backend.

35. Điều Quan Trọng Cần Nhớ

Nếu chỉ nhớ 10 ý, hãy nhớ:

1. Backend là nơi xử lý nghiệp vụ và bảo mật.
2. Frontend không được gọi database trực tiếp.
3. Controller nhận request, Service/UseCase xử lý nghiệp vụ, Repository làm việc với database.
4. DTO giúp API không phụ thuộc trực tiếp vào Entity và tránh lộ field nhạy cảm.
5. JWT dùng để xác thực request.
6. Role dùng để phân quyền STUDENT/LIBRARIAN/ADMIN.
7. Publication là đầu sách, Item là bản sao cụ thể.
8. Flyway quản lý schema database bằng migration version.
9. Kafka giúp gửi email/notification bất đồng bộ.
10. Backend gọi AI service, chứ không trực tiếp chạy model AI.